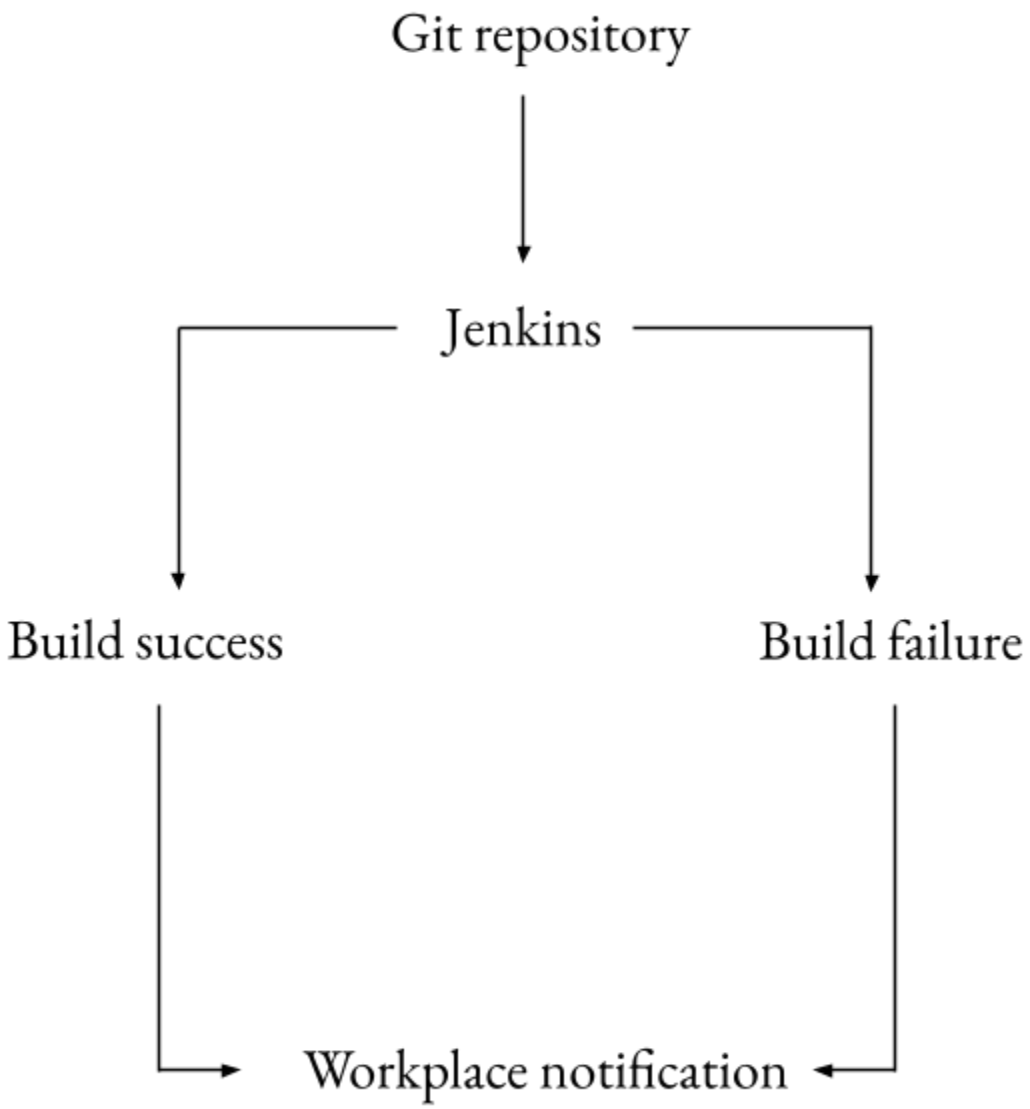
Workplace integration with Jenkins

This document was created by Suyesh Singh for Citytech global on Apr 14, 2022 .

Abstract

This is a process document for setting up jenkins and workplace so that a notification can be sent whenever a build is triggered in jenkins. Workplace receives a notification regarding the build status, namely success and failure.

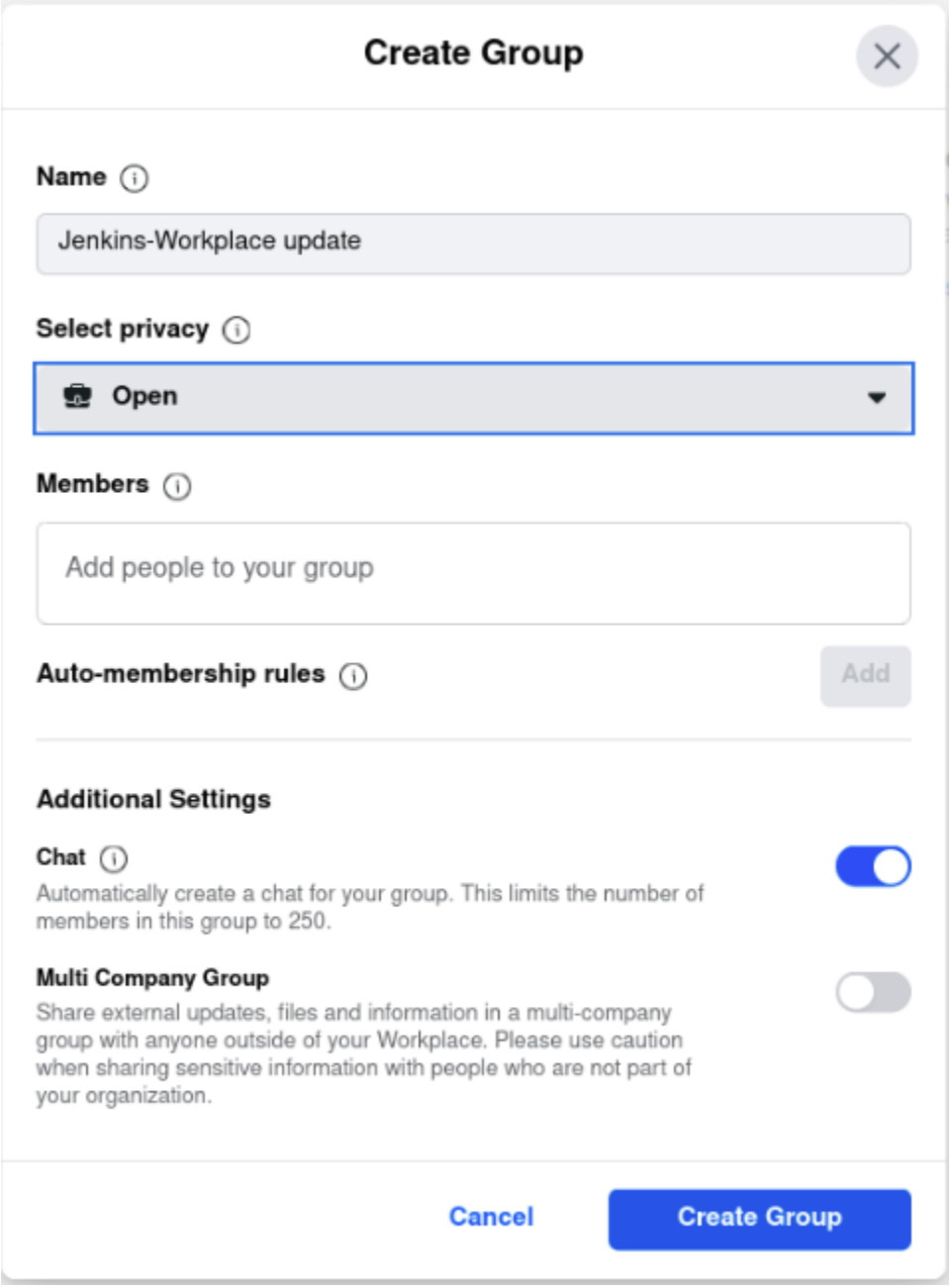


For the testing of this integration Github was used as a git repository and the current version of jenkins was 2.332.1.

1. Workplace setup

a. Setting up a workplace group to send notifications.

Step 1: First login to your workplace account as an administrator then go to *Groups* then click on *Create group.*

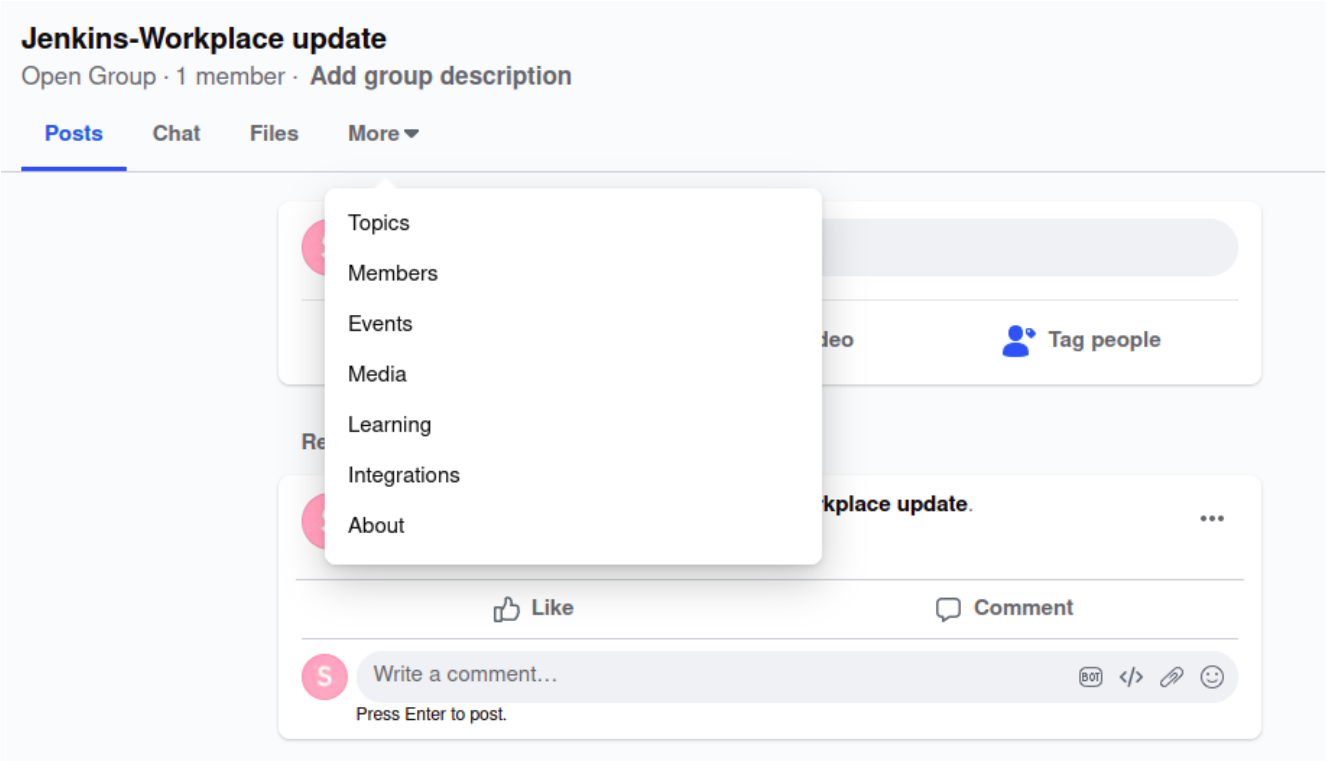


Step 2: Provide the name for your group and then set *Select Privacy* to *Open,* then click on *Create Group* Button.

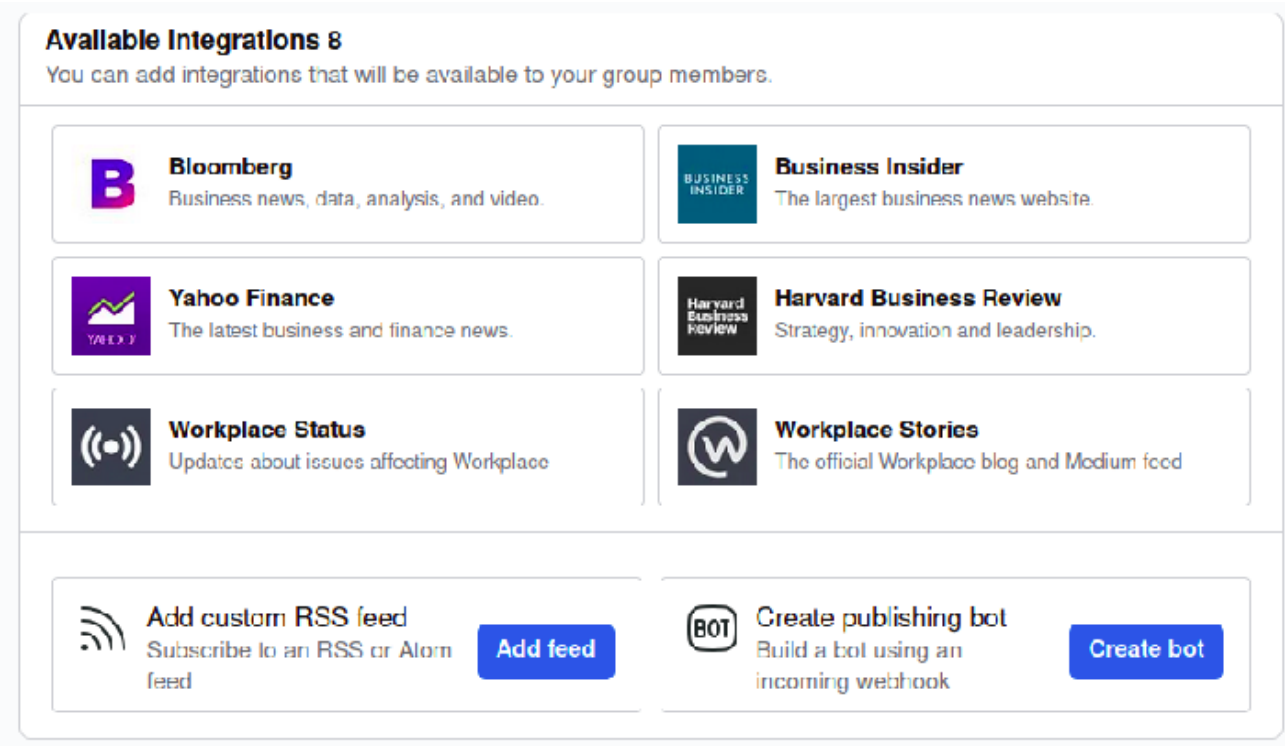
Now your group for pushing notifications is ready so the next step is to create a publishing bot.

b. Creating a publishing bot.

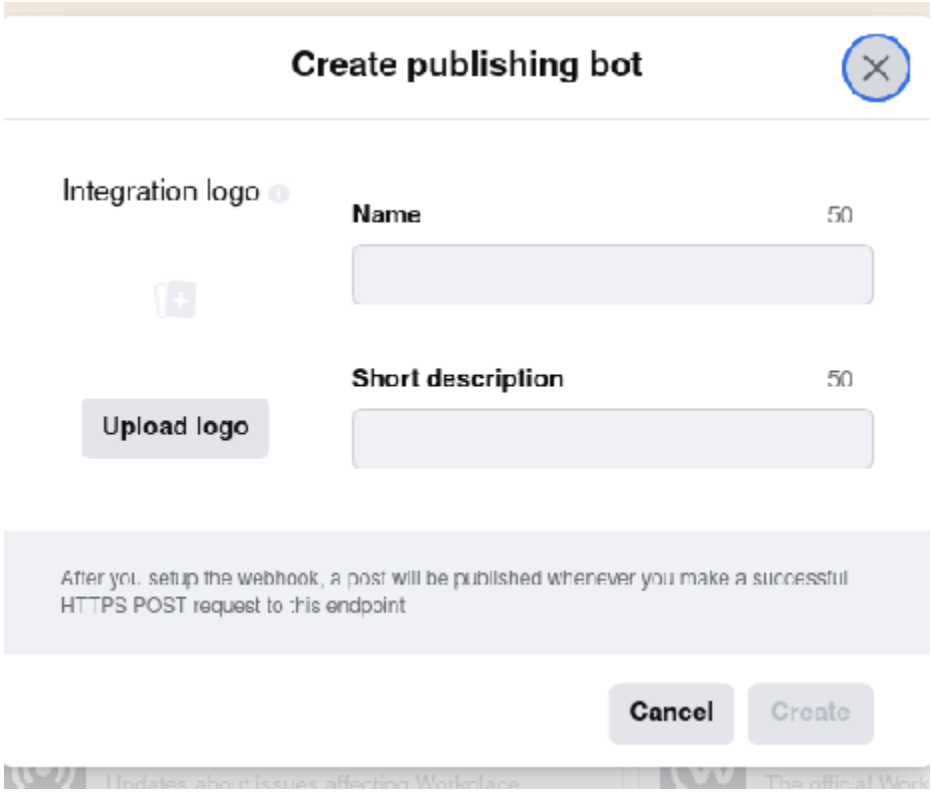
Step 1: In order to set up your publishing bot go to your group then click on *More* tab then click on *integrations*.



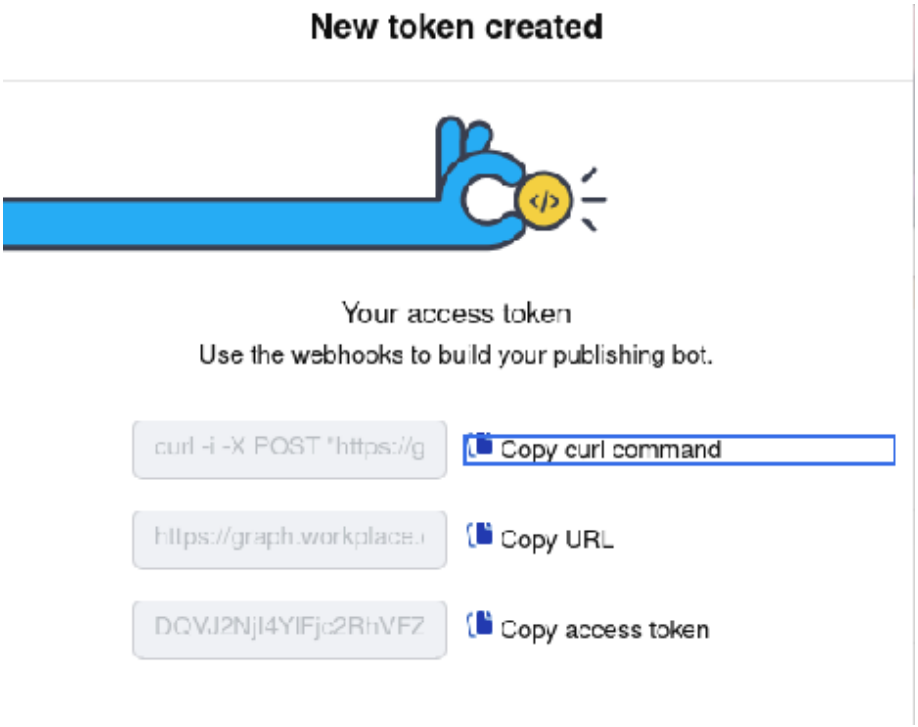
Step 2: After clicking on *Integrations* tab you should be able to see all the available integrations, There should be an option called *Create publishing bot,* In that option click on *Create bot* button.



Step 3: After clicking on *Create bot* button provide *Name, logo* and *description* of the bot.



Step 4: After providing all the information click on the create button, after that it is going to generate a token for the integration.

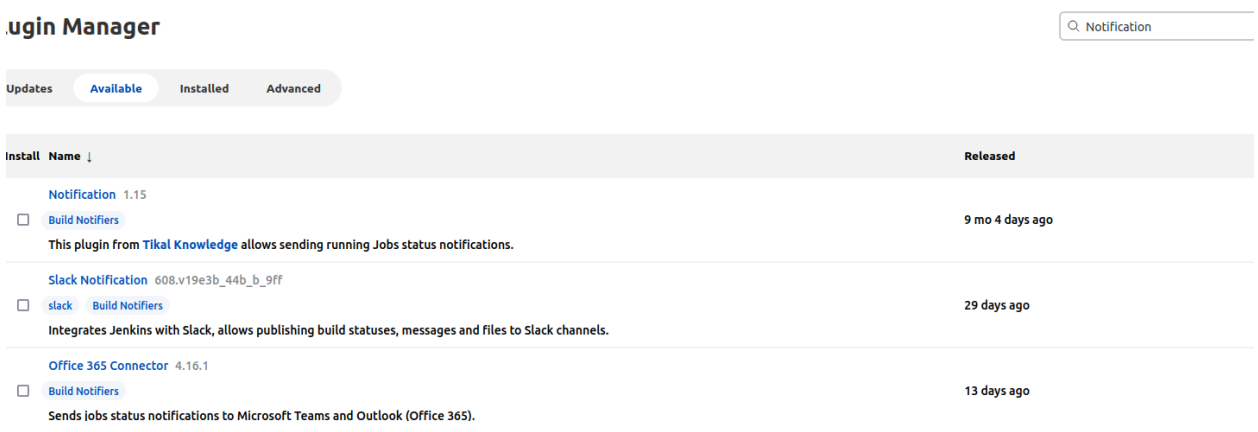


Note: Please make sure you save your access token as the same access token can not be generated again.

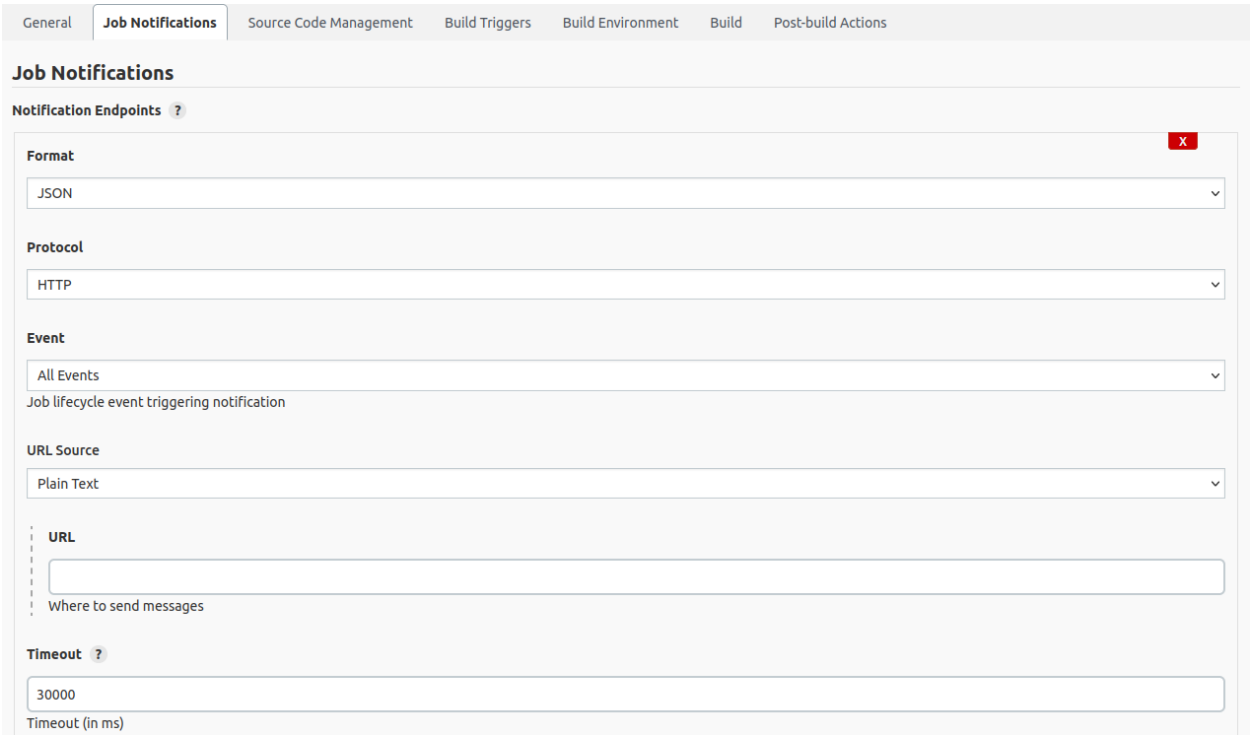
Now that we have our API access token we can get started with our Jenkins configuration.

2. Jenkins setup

Step 1: Go to jenkins plugin manager then click on *available* , then search for *Notification plugin.*



Step 2: Once you have downloaded the plugin, go to your project then go to the *Job Notifications* section.



Step 3: Now add your token generated when the publishing bot was created to the URL section.

The following endpoint URL is just an example and will not work for your environment.

https://graph.workplace.com/group/feed?message=Build%20success%20for%0ABuild%20name:%20${JOB_NAME}%0ABuild%20number:%20${BUILD_NUMBER}%0A${BUILD_USER}&access_token=DQVJ2UU9HRFhQU2pvdzIyNzlUUG5EdzN3c0lIcXVORHZAyOUtrbXZAEUVdueWRHUlIycVoxbmh6LTJNWnN0R3gwSllSaHBabm8tZATRNMlI3aExIdHExMkRzTFZABV040ckE3QVdwNEtOa2t4RTFQaDNQanBJOHduZAS1zLWVab21ydXdxTUVLTWhWR3NOX2FibWpaZAVNWdnFZAUnFIUnM0Q1BWYlVOVmc5NTMxb0NuaERENWE2dkdzSzdfRHh3dlhrOWlJT3NlSnln

%0A is used for space
%20 is used for line break

The following variables are available to shell and batch build steps:

BRANCH_NAME
For a multibranch project, this will be set to the name of the branch being built, for example in case you wish to deploy to production from master but not from feature branches; if corresponding to some kind of change request, the name is generally arbitrary (refer to CHANGE_ID and CHANGE_TARGET).
BRANCH_IS_PRIMARY
For a multibranch project, if the SCM source reports that the branch being built is a primary branch, this will be set to "true"; else unset. Some SCM sources may report more than one branch as a primary branch while others may not supply this information.
CHANGE_ID
For a multibranch project corresponding to some kind of change request, this will be set to the change ID, such as a pull request number, if supported; else unset.
CHANGE_URL
For a multibranch project corresponding to some kind of change request, this will be set to the change URL, if supported; else unset.
CHANGE_TITLE
For a multibranch project corresponding to some kind of change request, this will be set to the title of the change, if supported; else unset.
CHANGE_AUTHOR
For a multibranch project corresponding to some kind of change request, this will be set to the username of the author of the proposed change, if supported; else unset.

CHANGE_AUTHOR_DISPLAY_NAME
For a multibranch project corresponding to some kind of change request, this will be set to the human name of the author, if supported; else unset.
CHANGE_AUTHOR_EMAIL
For a multibranch project corresponding to some kind of change request, this will be set to the email address of the author, if supported; else unset.
CHANGE_TARGET
For a multibranch project corresponding to some kind of change request, this will be set to the target or base branch to which the change could be merged, if supported; else unset.
CHANGE_BRANCH
For a multibranch project corresponding to some kind of change request, this will be set to the name of the actual head on the source control system which may or may not be different from BRANCH_NAME. For example in GitHub or Bitbucket this would have the name of the origin branch whereas BRANCH_NAME would be something like PR-24.
CHANGE_FORK
For a multibranch project corresponding to some kind of change request, this will be set to the name of the forked repo if the change originates from one; else unset.
TAG_NAME
For a multibranch project corresponding to some kind of tag, this will be set to the name of the tag being built, if supported; else unset.
TAG_TIMESTAMP
For a multibranch project corresponding to some kind of tag, this will be set to a timestamp of the tag in milliseconds since Unix epoch, if supported; else unset.
TAG_UNIXTIME
For a multibranch project corresponding to some kind of tag, this will be set to a timestamp of the tag in seconds since Unix epoch, if supported; else unset.
TAG_DATE
For a multibranch project corresponding to some kind of tag, this will be set to a timestamp in the format as defined by java.util.Date#toString() (e.g., Wed Jan 1 00:00:00 UTC 2020), if supported; else unset.
JOB_DISPLAY_URL
URL that will redirect to a Job in a preferred user interface
RUN_DISPLAY_URL
URL that will redirect to a Build in a preferred user interface
RUN_ARTIFACTS_DISPLAY_URL
URL that will redirect to Artifacts of a Build in a preferred user interface
RUN_CHANGES_DISPLAY_URL
URL that will redirect to Changelog of a Build in a preferred user interface
RUN_TESTS_DISPLAY_URL
URL that will redirect to Test Results of a Build in a preferred user interface
CI
Statically set to the string "true" to indicate a "continuous integration" execution environment.

BUILD_NUMBER
The current build number, such as "153".
BUILD_ID
The current build ID, identical to BUILD_NUMBER for builds created in 1.597+, but a YYYY-MM-DD_hh-mm-ss timestamp for older builds.
BUILD_DISPLAY_NAME
The display name of the current build, which is something like "#153" by default.
JOB_NAME
Name of the project of this build, such as "foo" or "foo/bar".
JOB_BASE_NAME
Short Name of the project of this build stripping off folder paths, such as "foo" for "bar/foo".
BUILD_TAG
String of "jenkins-*${JOB_NAME}-${BUILD_NUMBER}*". All forward slashes ("/") in the JOB_NAME are replaced with dashes ("-"). Convenient to put into a resource file, a jar file, etc for easier identification.
EXECUTOR_NUMBER
The unique number that identifies the current executor (among executors of the same machine) that's carrying out this build. This is the number you see in the "build executor status", except that the number starts from 0, not 1.
NODE_NAME
Name of the agent if the build is on an agent, or "built-in" if run on the built-in node (or "master" until Jenkins 2.306).
NODE_LABELS
Whitespace-separated list of labels that the node is assigned.
WORKSPACE
The absolute path of the directory assigned to the build as a workspace.
WORKSPACE_TMP
A temporary directory near the workspace that will not be browsable and will not interfere with SCM checkouts. May not initially exist, so be sure to create the directory as needed (e.g., mkdir -p on Linux). Not defined when the regular workspace is a drive root.
JENKINS_HOME
The absolute path of the directory assigned on the controller file system for Jenkins to store data.
JENKINS_URL
Full URL of Jenkins, like http://server:port/jenkins/ (note: only available if *Jenkins URL* set in system configuration).
BUILD_URL
Full URL of this build, like http://server:port/jenkins/job/foo/15/ (*Jenkins URL* must be set).
JOB_URL
Full URL of this job, like http://server:port/jenkins/job/foo/ (*Jenkins URL* must be set).
GIT_COMMIT
The commit hash being checked out.
GIT_PREVIOUS_COMMIT
The hash of the commit last built on this branch, if any.
GIT_PREVIOUS_SUCCESSFUL_COMMIT

The hash of the commit last successfully built on this branch, if any.

GIT_BRANCH

The remote branch name, if any.

GIT_LOCAL_BRANCH

The local branch name being checked out, if applicable.

GIT_CHECKOUT_DIR

The directory that the repository will be checked out to. This contains the value set in Checkout to a sub-directory, if used.

GIT_URL

The remote URL. If there are multiple, will be GIT_URL_1, GIT_URL_2, etc.

GIT_COMMITTER_NAME

The configured Git committer name, if any, that will be used for FUTURE commits from the current workspace. It is read from the **Global Config user.name Value** field of the Jenkins **Configure System** page.

GIT_AUTHOR_NAME

The configured Git author name, if any, that will be used for FUTURE commits from the current workspace. It is read from the **Global Config user.name Value** field of the Jenkins **Configure System** page.

GIT_COMMITTER_EMAIL

The configured Git committer email, if any, that will be used for FUTURE commits from the current workspace. It is read from the **Global Config user.email Value** field of the Jenkins **Configure System** page.

GIT_AUTHOR_EMAIL

The configured Git author email, if any, that will be used for FUTURE commits from the current workspace. It is read from the **Global Config user.email Value** field of the Jenkins **Configure System** page.

MERCURIAL_REVISION

Full ID of revision checked out.

MERCURIAL_REVISION_SHORT

Abbreviated ID of revision checked out.

MERCURIAL_REVISION_NUMBER

Number of revision checked out (not portable across clones).

MERCURIAL_REVISION_BRANCH

Branch of revision checked out, if not checking out by branch head.

MERCURIAL_REPOSITORY_URL

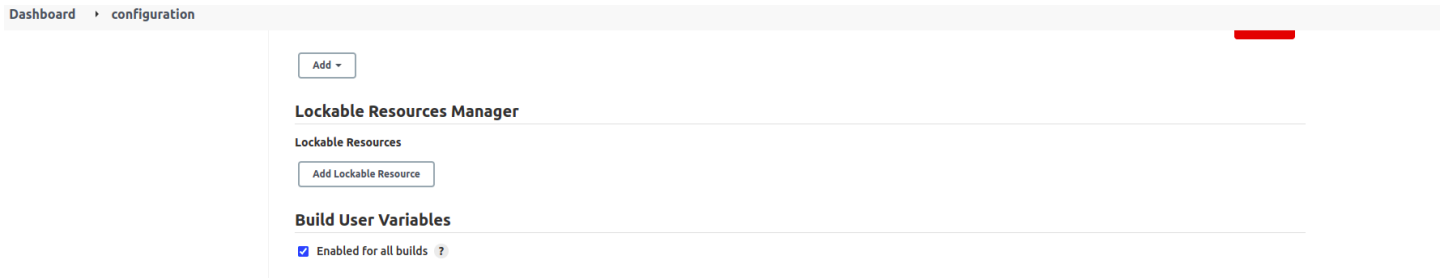
URL of repository.

Step 4: Adding environment variables that describe the user who started the build.

Now, in order to add certain environment variables that allow us to display username and email we are going to install a plugin called *build user var*.

The plugin can be downloaded for manage plugins section or the following link:
https://plugins.jenkins.io/build-user-vars-plugin/

After the plugin is downloaded restart your jenkins server then go to *manage jenkins* then to *configure system*. Once there look for *Build User Variables* then enable the Build user var.



Now we should be able to use the following additional variables.

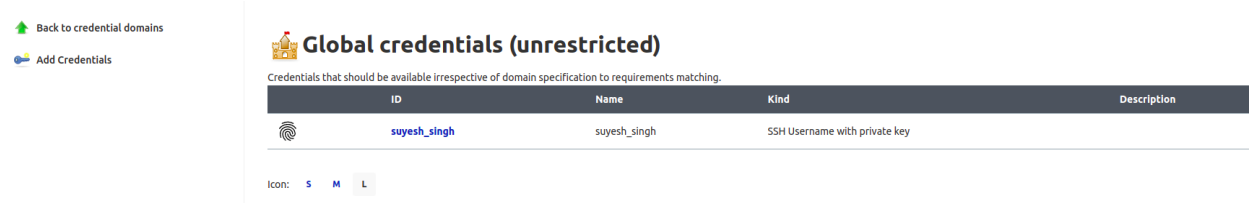
Variable	Description
BUILD_USER	Full name (first name + last name)
BUILD_USER_FIRST_NAME	First name
BUILD_USER_LAST_NAME	Last name
BUILD_USER_ID	Jenkins user ID
BUILD_USER_GROUPS	Jenkins user groups
BUILD_USER_EMAIL	Email address

Step 5: Hiding endpoint URL

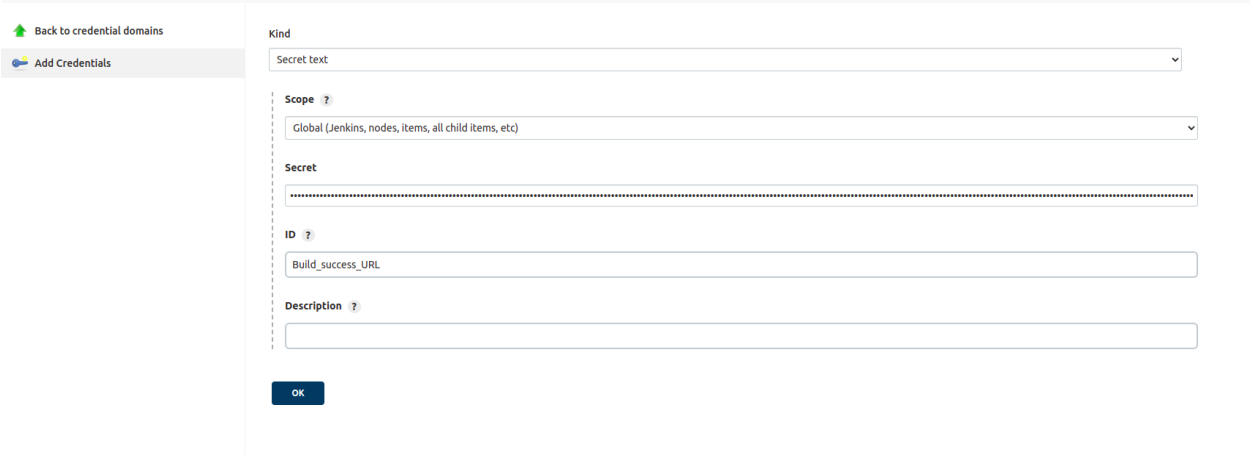
Now, you have your api token, endpoint url and you have configured all the environment variables you want to use. The final step is to hide that endpoint url so it isn't in a plain text format.

Note: Please make sure that your endpoint url works correctly and shows all the required variables before hiding it as it cannot be modified once it is hidden.

First copy your endpoint URL then go to *manage credentials* , then go to your *Global credential* then click on *add credentials.*



Now select the credential as secret text then paste your endpoint URL, provide description so that you know what the secret is for.

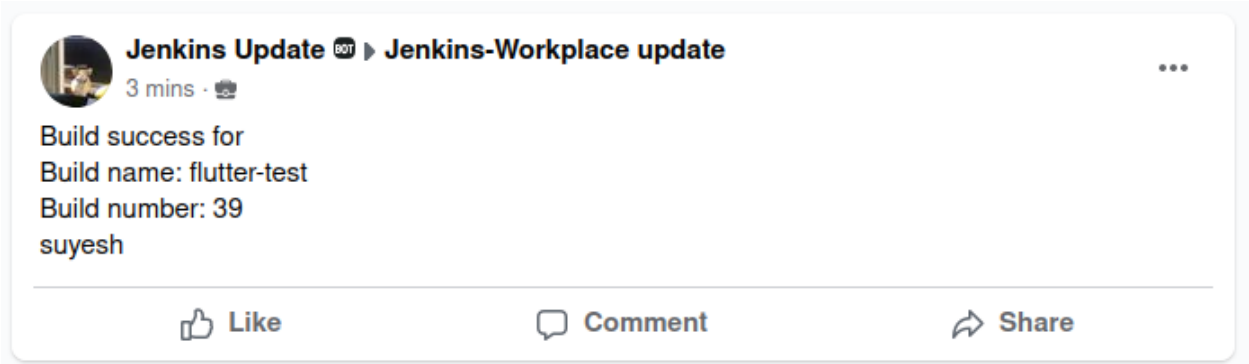


Now click on ok. Now we can use this without having to expose our token in plain text.

Now once you build your project you should see the following line in your console output

Notifying endpoint with credentials id 'build_success'

Now we can check our workplace to check if we received our notification or not.



Now, we can see we have received our notification in our workplace group regarding the jenkins build which concludes this project.